

Software Test Plan (STP): Snake Game

Project: Snake Game (graphics or console-based interface)

Version: 1.0

Authors: S Prithika(PES2UG24CS415), S N Niharika(PES2UG24CS414), S V Bhavana(PES2UG24CS417),
S Ashwin(PES2UG24CS413)

Team: 11

Date: 17-09-2026

Status: Draft

1. Introduction

Purpose

This document defines the test plan for the Snake Game. It outlines the objectives, scope, strategy, resources, schedule, and responsibilities for testing the system as specified in the Software Requirements Specification (SRS), Team 11.

Scope

Testing covers all functional areas of the Snake Game described in the SRS: main menu and initialization, movement and control, food and growth, collision and game over, scoring, pause/resume, difficulty selection, and Administrator authentication/authorization. Testing also covers the associated non-functional requirements (performance, usability, reliability) and security requirements (input validation, unauthorized access/modification prevention). Network communication, multiplayer functionality, and external service integration are explicitly out of scope, as the system is a standalone, offline, single-player application.

References

- Software Requirements Specification (SRS) - Snake Game, Team 11, 04-09-2026
- UML Use-Case Diagrams (UML1: Player use cases; UML2: Administrator use cases)
- Requirements Traceability Matrix (RTM), SRS Section 8

Definitions

Snake, Food, Game Board, Game Tick, Score, Game Over, Difficulty, HUD, FR (Functional Requirement), NFR (Non-Functional Requirement), UML (Unified Modeling Language), RTM (Requirements Traceability Matrix) as defined in SRS Section 1.4.

2. Test Items

The following modules, as defined in SRS Section 2, are subject to test:

- MenuModule - Main Menu, difficulty selection, navigation
- RenderModule / UI - HUD, Game Over screen, on-screen redraws
- InputModule - keyboard input and directional control
- GameCore - snake position, food, score, collisions, game state

- PersistenceModule - stored score and configuration data
- AdminModule - Administrator authentication and administrative operations
- ValidationModule - input and data validation

3. Features to be Tested

Features mapped to SRS requirement IDs:

- SNK-F-001 – Display Main Menu on launch
- SNK-F-002 – Difficulty level selection
- SNK-F-003 – Game/board/snake initialization
- SNK-F-004 – Directional input, 180° reversal rejection
- SNK-F-005 – Redraw snake/grid every tick
- SNK-F-006 – Exit / return to Main Menu via keyboard
- SNK-F-007 – Food spawn on unoccupied cells
- SNK-F-008 – Snake growth and food respawn on consumption
- SNK-F-009 – Wall / self-collision detection
- SNK-F-010 – Game Over screen with Restart/Exit
- SNK-F-011 – Score increase on food consumption
- SNK-F-012 – Final score display at game end
- SNK-F-013 – Pause / Resume without state loss
- SNK-F-014 – Difficulty-based movement speed
- SNK-F-015 – Administrator authentication
- SNK-F-016 – Authenticated Administrator access to protected data
- SNK-F-017 – Authenticated Administrator modification of stored score/config
- SNK-F-018 – Authenticated Administrator administrative/configuration operations
- SNK-NF-001 – Directional input processed ≤ 100 ms
- SNK-NF-002 – Keyboard-only menu/difficulty navigation
- SNK-NF-003 – Stable FPS / tick rate (≥ 30 FPS or consistent tick interval)
- SNK-NF-004 – 30-minute continuous run without crash/leak
- SNK-NF-005 – Input-to-movement response ≤ 200 ms
- SNK-NF-006 – Clearly distinguishable controls/status info
- SNK-SR-001 – Validation of player input/configuration values
- SNK-SR-002 – Prevention of unauthorized score modification
- SNK-SR-003 – Restriction of admin/config operations to authorized users
- SNK-SR-004 – Protection of stored data from unauthorized access/disclosure

4. Features Not to be Tested

- Network communication, multiplayer functionality, or external service integration (explicitly out of scope per SRS Section 1.2 and 3.4)
- Operating-system keyboard/input API internals and third-party rendering/game-loop library internals (assumed correct per vendor/library documentation)
- Hardware-level display and keyboard device behavior beyond the application's input handling layer

5. Test Approach / Strategy

Levels

- Unit tests (module-level: GameCore, InputModule, ValidationModule, AdminModule, etc.)
- Integration tests (e.g., InputModule - GameCore, AdminModule - PersistenceModule)
- System tests (end-to-end gameplay: menu - play - game over)
- Acceptance tests (per acceptance criteria defined in SRS Sections 4–5)

Types

- Functional testing (menu, movement, food/growth, collision, scoring, pause/resume, difficulty, admin functions)
- Regression testing (after each build/fix cycle)
- Performance testing (input latency, frame/tick rate, response time)
- Usability testing (keyboard-only navigation, control clarity)
- Reliability testing (30-minute soak test)
- Security testing (input validation, unauthorized access/modification attempts)

Entry Criteria

Stable build delivered; test environment set up (development/runtime environment and rendering library available); test data (dummy scores/configuration) available; SRS and RTM baselined.

Exit Criteria

100% of planned test cases executed; 0 critical/high-severity defects open; all acceptance criteria in SRS Sections 4–6 satisfied; RTM shows full requirement coverage.

5.1 Security Validation

- Validate rejection of invalid, excessively long, or unsupported player/configuration input (SNK-SR-001)
- Verify unauthorized score modification attempts are rejected and original data preserved (SNK-SR-002)
- Verify administrative/configuration operations are restricted to authenticated Administrators (SNK-SR-003, SNK-F-015)
- Verify stored player/configuration data is not accessible or disclosed to unauthorized users (SNK-SR-004)
- Attempt authentication bypass and boundary/negative-value testing on stored score data

6. Test Environment

Hardware

Desktop or laptop computer with a display supporting a minimum resolution of 800×600 pixels and a keyboard, per SRS Section 2.4.

Software

Supported operating system and runtime environment; selected graphics/rendering or game-loop library (e.g., SFML for graphics mode, or console I/O for console mode).

Tools

- Manual and scripted test execution (keyboard-driven test scripts)

- Frame-time / tick-interval logging utility (performance testing)
- Memory profiler (reliability / soak testing)
- Defect tracking tool (e.g. Jira or equivalent)

Test Data

Dummy Administrator credentials, sample stored score/configuration records, and a range of valid/invalid/boundary input values for configuration fields.

7. Test Schedule

Milestones:

- Test case design: 22-Sep-2026
- Environment setup: 24-Sep-2026
- Test execution start: 25-Sep-2026
- Test execution end: 8-Oct-2026
- Acceptance: 10-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (mapped to SRS requirement IDs, per Sections 6.1–6.3 of the SRS)
- Test Scripts (manual procedures / any automated scripts)
- Test Data sets
- Test Execution Logs
- Defect Reports
- Requirements Traceability Matrix (updated with pass/fail status)
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	S N Niharika	Prepare test plan, coordinate execution
Test Engineer	S V Bhavana	Design & execute test cases, log defects
Developer	S Ashwin	Support defect fixes and triage
Product Owner	S Prithika	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early smoke builds from the development team
Rendering library / environment instability	Maintain a fallback console-mode build for testing core logic

Risk	Mitigation
Ambiguity in timing-based NFRs (e.g., tick rate) across hardware	Run performance tests on a standard reference machine and document specs
Limited team availability for full regression coverage	Prioritize high-priority requirements (SNK-F-001/003/007-010/015) first

11. Assumptions & Dependencies

- The selected rendering/game-loop library and OS input APIs behave as documented
- Test data (dummy Administrator credentials, sample scores/config) will be available before execution
- No network, multiplayer, or external service testing is required, consistent with SRS scope
- A stable, feature-complete build will be delivered ahead of each test cycle

12. Suspension & Resumption Criteria

Suspend testing if:

- The build is too unstable and blocks more than 30% of planned test cases
- The test environment is unavailable for more than 4 hours

Resume testing if:

- Blocking defects are resolved
- The environment is stabilized and re-verified

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM), as defined in SRS Section 8, maps every SRS requirement to its corresponding test case(s). Representative mappings:

Req ID	Requirement	Test Case(s)
SNK-F-001	Display Main Menu on launch	TC-Menu-01
SNK-F-004	Directional input, reject 180° reversal	TC-Move-02, TC-Ctrl-02
SNK-F-007/008	Food spawn / growth / respawn	TC-Food-01, TC-Food-02, TC-Food-03
SNK-F-009/010	Collision detection / Game Over screen	TC-Collision-01, -02, -03
SNK-F-015..018	Administrator authentication & operations	TC-Admin-01..04
SNK-NF-001	Input latency ≤100 ms	TC-Perf-Ctrl-01
SNK-NF-003/004	Stable frame rate / 30-min stability	TC-Perf-01, TC-Rel-01
SNK-SR-001..004	Input validation / data protection	TC-Sec-01..05

The full RTM (all 28 requirement IDs) is maintained in the SRS, Section 8, and shall be updated with Pass/Fail/Not-Run status throughout execution.

14. Test Metrics & Reporting

Metrics collected

- % of test cases executed
- % passed / failed
- Defect density and defect aging
- Requirement coverage (from the RTM)
- Input latency and frame/tick-rate measurements against SNK-NF-001/003/005

Reports

- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	S N Niharika	S N Niharika - 12/09/26
Dev Lead	S V Bhavana	S V BHAVANA - 12/09/26
Product Owner	S Prithika	S Prithika -12/09/26